1. **CHATGPT and CLAUDE**
2. **I accessed ChatGPT in the Chrome browser and Claude in the computer application**
3. **PROMPT →**

Assignment:
1. Use two GenAI's to generate C++ code using objects, not functions, to implement the following requirements (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.):
• Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
• A typical puzzle is shown here (image credit: Wikimedia Commons):
o
• Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
o Each digit from 1 to 9 must appear exactly once in each row.
o Each digit from 1 to 9 must appear exactly once in each column.
o Each digit from 1 to 9 must appear exactly once in each block.
• Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
• The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
• It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it's possible to gradually fill in the entire grid.
• That is not the approach we're going to use. Instead, we're going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by "slower to run," we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
• Here's the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we

get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).
• This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
• You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
• Write a recursive object-oriented program to find the solution to the five puzzle files.
• The output for each solution, should include:
o The puzzle file name (e.g., puzzle1.txt)
o The puzzle from the file printed out.
o The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.
• The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
• To help you debug your program, solution1.txt is a solution to puzzle1.txt

4. CHATGPT:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

class Sudoku {
private:
    int board[9][9];

    // Determine whether a number can be placed in a cell.
    bool isValid(int row, int col, int number) {
        // Check the row and column.
        for (int i = 0; i < 9; i++) {
            if (board[row][i] == number ||
                board[i][col] == number) {
                return false;
            }
        }
```

```
    // Find the starting position of the 3x3 block.
    int startRow = (row / 3) * 3;
    int startCol = (col / 3) * 3;

    // Check the 3x3 block.
    for (int r = startRow; r < startRow + 3; r++) {
        for (int c = startCol; c < startCol + 3; c++) {
            if (board[r][c] == number) {
                return false;
            }
        }
    }

    return true;
}

// Recursively find and print all solutions.
void solveRecursive(int row, int col, int &solutions) {
    // Reaching row 9 means the entire puzzle is filled.
    if (row == 9) {
        solutions++;
        cout << "Solution " << solutions << ":\n";
        printBoard();
        return;
    }

    // Determine which cell comes next.
    int nextRow = row;
    int nextCol = col + 1;

    if (nextCol == 9) {
        nextRow++;
        nextCol = 0;
    }

    // Skip cells containing original numbers.
    if (board[row][col] != 0) {
        solveRecursive(nextRow, nextCol, solutions);
        return;
    }

    // Try each digit from 1 through 9.
    for (int number = 1; number <= 9; number++) {
        if (isValid(row, col, number)) {
```

```
            // Place a valid candidate.
            board[row][col] = number;

            // Recursively attempt to finish the puzzle.
            solveRecursive(nextRow, nextCol, solutions);

            // Undo the placement to try other candidates.
            board[row][col] = 0;
        }
    }
  }

public:
  // Initialize every cell to zero.
  Sudoku() {
    for (int row = 0; row < 9; row++) {
      for (int col = 0; col < 9; col++) {
        board[row][col] = 0;
      }
    }
  }

  // Read and validate a puzzle file.
  bool loadPuzzle(const string &filename) {
    ifstream input(filename);

    if (!input.is_open()) {
      cout << "Error opening " << filename << endl;
      return false;
    }

    char value;

    for (int row = 0; row < 9; row++) {
      for (int col = 0; col < 9; col++) {
        if (!(input >> value)) {
          cout << "Invalid puzzle file: "
               << filename << endl;
          return false;
        }

        if (value == '_') {
          board[row][col] = 0;
        }
```

```
            else if (value >= '1' && value <= '9') {
                int number = value - '0';

                if (!isValid(row, col, number)) {
                    cout << "Invalid puzzle: "
                         << filename << endl;
                    return false;
                }

                board[row][col] = number;
            }
            else {
                cout << "Invalid character in "
                     << filename << endl;
                return false;
            }
        }
    }

    // Reject additional non-whitespace characters.
    if (input >> value) {
        cout << "Invalid puzzle file: "
             << filename << endl;
        return false;
    }

    return true;
}

// Display the current Sudoku grid.
void printBoard() {
    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            if (board[row][col] == 0) {
                cout << "_ ";
            }
            else {
                cout << board[row][col] << " ";
            }

            if (col == 2 || col == 5) {
                cout << "| ";
            }
        }
```

```
            cout << endl;

            if (row == 2 || row == 5) {
                cout << "------+-------+------\n";
            }
        }

        cout << endl;
    }

    // Begin the recursive solving process.
    void solve() {
        int solutions = 0;

        solveRecursive(0, 0, solutions);

        if (solutions == 0) {
            cout << "No solution found" << endl;
        }
        else {
            cout << "Total solutions found: "
                 << solutions << endl;
        }
    }
};

int main() {
    // Process each of the five Sudoku puzzles.
    for (int i = 1; i <= 5; i++) {
        string filename = "puzzle" + to_string(i) + ".txt";

        cout << "\n================================\n";
        cout << "Puzzle File: " << filename << endl;
        cout << "================================\n";

        // Create a separate Sudoku object for each puzzle.
        Sudoku puzzle;

        if (puzzle.loadPuzzle(filename)) {
            cout << "\nOriginal Puzzle:\n";
            puzzle.printBoard();

            cout << "Solutions:\n";
```

```
            puzzle.solve();
        }
    }

    return 0;
}
```

CLAUDE:

```
// Sudoku brute-force solver: depth-first search with backtracking.
// Object-oriented: Puzzle holds the grid, Solver does the recursive search.
// Finds ALL solutions. Usage: ./sudoku [file1 file2 ...]
// With no arguments it solves puzzle1.txt ... puzzle5.txt.

#include <fstream>
#include <iostream>
#include <string>
#include <vector>

class Puzzle {
public:
    static const int N = 9;

    Puzzle() : grid_(N, std::vector<int>(N, 0)) {}

    // Reads 81 cells: digits 1-9 or '_' for blank. Whitespace is ignored.
    bool load(const std::string& filename) {
        std::ifstream in(filename);
        if (!in) return false;
        int count = 0;
        char ch;
        while (in.get(ch) && count < N * N) {
            if (ch >= '1' && ch <= '9')      grid_[count / N][count % N] = ch - '0';
            else if (ch == '_' || ch == '0' || ch == '.') grid_[count / N][count % N] = 0;
            else continue;  // skip whitespace / other characters
            ++count;
        }
        return count == N * N;
    }

    int get(int r, int c) const { return grid_[r][c]; }
    void set(int r, int c, int v) { grid_[r][c] = v; }

    // True if placing v at (r,c) breaks no row, column, or block constraint.
```

```
    bool canPlace(int r, int c, int v) const {
        for (int i = 0; i < N; ++i) {
            if (grid_[r][i] == v) return false;
            if (grid_[i][c] == v) return false;
        }
        int br = (r / 3) * 3, bc = (c / 3) * 3;
        for (int i = 0; i < 3; ++i)
            for (int j = 0; j < 3; ++j)
                if (grid_[br + i][bc + j] == v) return false;
        return true;
    }

    // Checks the given clues don't already conflict with each other.
    bool givensConsistent() {
        for (int r = 0; r < N; ++r)
            for (int c = 0; c < N; ++c) {
                int v = grid_[r][c];
                if (v == 0) continue;
                grid_[r][c] = 0;
                bool ok = canPlace(r, c, v);
                grid_[r][c] = v;
                if (!ok) return false;
            }
        return true;
    }

    void print(std::ostream& out = std::cout) const {
        for (int r = 0; r < N; ++r) {
            for (int c = 0; c < N; ++c) {
                out << (grid_[r][c] ? char('0' + grid_[r][c]) : '_');
                if (c < N - 1) out << ' ';
            }
            out << '\n';
        }
    }

private:
    std::vector<std::vector<int>> grid_;
};

class Solver {
public:
    explicit Solver(const Puzzle& p) : puzzle_(p) {}
```

```
    // Runs the search and returns every solution found.
    const std::vector<Puzzle>& solveAll() {
        solutions_.clear();
        if (puzzle_.givensConsistent()) search();
        return solutions_;
    }

private:
    Puzzle puzzle_;
    std::vector<Puzzle> solutions_;

    // Recursive DFS with backtracking. Picks the empty cell with the fewest
    // candidates (still exhaustive, so all solutions are found).
    void search() {
        int bestR = -1, bestC = -1, bestCount = 10;
        std::vector<int> bestCands;

        for (int r = 0; r < Puzzle::N; ++r) {
            for (int c = 0; c < Puzzle::N; ++c) {
                if (puzzle_.get(r, c) != 0) continue;
                std::vector<int> cands;
                for (int v = 1; v <= 9; ++v)
                    if (puzzle_.canPlace(r, c, v)) cands.push_back(v);
                if (cands.empty()) return;  // dead end: backtrack
                if ((int)cands.size() < bestCount) {
                    bestCount = (int)cands.size();
                    bestR = r; bestC = c;
                    bestCands = cands;
                }
            }
        }

        if (bestR == -1) {  // no empty cells: all 81 filled
            solutions_.push_back(puzzle_);
            return;
        }

        for (int v : bestCands) {
            puzzle_.set(bestR, bestC, v);
            search();                   // recursive call
            puzzle_.set(bestR, bestC, 0);   // undo (backtrack)
        }
    }
};
```

```
int main(int argc, char* argv[]) {
    std::vector<std::string> files;
    if (argc > 1) {
        for (int i = 1; i < argc; ++i) files.push_back(argv[i]);
    } else {
        for (int i = 1; i <= 5; ++i) files.push_back("puzzle" + std::to_string(i) + ".txt");
    }

    for (const std::string& name : files) {
        std::cout << "==============================\n";
        std::cout << name << "\n\n";

        Puzzle puzzle;
        if (!puzzle.load(name)) {
            std::cout << "Could not read a valid puzzle from this file.\n\n";
            continue;
        }

        std::cout << "Puzzle:\n";
        puzzle.print();
        std::cout << '\n';

        Solver solver(puzzle);
        const std::vector<Puzzle>& solutions = solver.solveAll();

        if (solutions.empty()) {
            std::cout << "No solution found\n\n";
        } else {
            for (size_t i = 0; i < solutions.size(); ++i) {
                if (solutions.size() > 1)
                    std::cout << "Solution " << (i + 1) << " of " << solutions.size() << ":\n";
                else
                    std::cout << "Solution:\n";
                solutions[i].print();
                std::cout << '\n';
            }
        }
    }
    return 0;
}
```

5.

Time Complexity: Both codes use brute-force backtracking, meaning they simply guess and check possible solutions until they find all possible solutions. Because of this, they both have incredibly inefficient worst-case time complexity. However, Claude uses a heuristic, a strategy that will help the code find a solution faster. In this case, it is a Minimum Remaining Values strategy. This means Claude's code will first try to solve the cells that have the fewest possible valid numbers and will then work from there. While this does not always guarantee a quicker result, it definitely can help lower the work the algorithm has to do. This is because we can more quickly rule out incorrect solutions, minimizing the number of times we have to backtrack all the way to the start. ChatGPT's code does not implement this heuristic, and rather works in sequential order when solving each cell.

Space Complexity: The main difference in space complexity when comparing the two codes stems from what the program does with the found solutions. While ChatGPT simply prints out all found solutions, Claude stores each one. This means that it will use more storage than ChatGPT. A smaller, more negligible difference is that ChatGPT utilizes a fixed array, while Claude decided to implement a dynamic array. While they have the same basic memory size, dynamic arrays use extra storage to maintain the vector's size, allocated memory, and capacity.

Maintainability: Claude organizes its code into two classes, Puzzle and Solver. Meanwhile, ChatGPT has a more straightforward approach by having only one Sudoku class. While this simplicity has its own advantages for maintainability, it is less organized than Claude's two-class system. In this case, I think that the more organized approach is more maintainable, as each class has its own responsibilities and purpose (separation of concerns), making it easy to make changes. Additionally, because of the two different classes, Claude's Puzzle class is able to be used with other Sudoku solvers. This reusability is beneficial to the developers and the program's maintainability.

Correctness: Both programs demonstrate good correctness and satisfy the requirements for the project. However, they differ slightly when it comes to how the algorithm handles invalid input. For example, ChatGPT is more prone to rejecting invalid data. More specifically, it will reject unexpected characters, while Claude's code simply skips past them. Additionally, ChatGPT's code also checks for extra data in the puzzle files, while Claude makes no effort to check for data past the first 81 expected cells. Because of these differences in input validation, I think that ChatGPT's algorithm demonstrates better correctness.

6. I chose the Claude code because it excels in aspects that will be harder to modify in the ChatGPT code. For example, I think Claude's main advantage is the two-class system. To improve the organization in the ChatGPT version, you would have to restructure the algorithm. Meanwhile, it will be easy to make changes to

the Claude code to strengthen its weaknesses, such as invalid input handling! Compared to ChatGPT's code, Claude's exhibits stronger groundwork to create a more efficient and maintainable program!

7.

Time Complexity: The first thing I would change is to optimize the Minimum Remaining Values heuristic so that it doesn't always need to cycle through every cell to find the MRV. For example, once the algorithm finds a cell that has only one possible value, it will stop searching for a better cell. The next big change I would make is to use bitmasks to find which numbers are unavailable per 3x3 block, row, and column. This would allow us to turn loops into a couple of bitwise logical operations.

Space Complexity: Since the ChatGPT code used less memory, I would make changes to the Claude code that makes it resemble the ChatGPT code more. More specifically, I would first stop storing all solutions and rather just print them right after they are found. This way, we don't have to waste storage by storing multiple Sudoku boards. Additionally, I would change the dynamic vector to a fixed array, saving memory space because of the extra memory usage that comes with dynamic vectors.

Maintainability: Firstly, I would make the variable names more clear. For example, there are many variable names that are short and abbreviated, such as br, bc, and v. Making these names more clear (like to blockRow, blockCol, and value) will make it easier to read and debug. Additionally, the search() method currently handles a lot of responsibilities. I think we could do a better job applying decomposition by making some more smaller, more defined methods, such as findBestCell(). Finally, I would add more comments to specify the purpose of each line/block of code!

Correctness: As mentioned earlier, Claude's weaknesses when it comes to correctness mainly have to deal with bad input handling. To fix this, I would first make the code reject unexpected characters, rather than simply skipping past and ignoring them. Furthermore, I would make it so it checks for extra input, beyond the expected 81 cells.